JamJamBeat용 테스트 모델 비교 정리

기준일: 2026-03-10. 아래 추천은 이 저장소의 현재 구조를 보고 정리한 실험 우선순위다.

> **v2 변경사항 (2026-03-10)**
> - 전처리 파이프라인(`hand_gesture_preprocess.py`) 기반 피처 체계 반영
> - Two-Stream (Joint + Bone) 구조 추가
> - 피처 조합 × 모델 실험 매트릭스 신설 (섹션 3.1)
> - 공통 평가 체계 신설 (섹션 4)
> - Focal Loss + class-balanced sampling 기본 적용으로 변경
> - 실험 순서 업데이트 (섹션 8)

1. 현재 프로젝트 기준 입력 구조

- 현재 저장소는 MediaPipe 손 랜드마크 기반 파이프라인이다.
- 프레임당 원본 입력은 `21 landmarks x (x, y, z) = 63차원`이다.
- 시퀀스 실험은 `src/dataset/sliding_window.py` 기준으로 `16 x D` 형태가 자연스럽다.
- 로컬에 이미 `hand_landmarker.task`가 있고, 파일 크기는 현재 약 `7.5MB`다.
- `data/landmark_data` 기준 현재 로컬 데이터는 `28개 CSV / 24,753 frame / 66 columns`다.

1.1 전처리 후 피처 체계 (신규)

`hand_gesture_preprocess.py`를 거치면 프레임당 피처가 아래와 같이 확장된다.

피처	차원	내용	출력 컬럼 접두어
Joint (정규화 좌표)	63	Translation/Scale/Rotation 정규화된 21점 × (x,y,z)	`nx, ny, nz`
Bone (뼈대 벡터+길이)	84	HAND_CONNECTIONS 21개 엣지 × (dx,dy,dz,length)	`bx, by, bz, bl`
Angle (굽힘+벌어짐)	9	5개 finger flexion + 4개 finger abduction	`flex_*, abd_*`
합계	**156**		

정규화 기준:

- **Translation**: index(5)/middle(9)/pinky(17) knuckle 평균을 원점으로 이동
- **Scale**: center에서 가장 먼 knuckle까지 거리로 나눔
- **Rotation**: alignment vector(middle knuckle→wrist + index→pinky)를 기준축으로 2D 회전

이 정규화는 촬영 거리, 손 크기, 카메라 앞 회전 변화에 대한 분산을 줄여서 모든 모델의 베이스라인 성능을 올리는 역할을 한다.

1.2 결측 프레임 처리

원본 CSV에서 landmark가 NaN인 행(=손 미검출)은 두 종류로 나뉜다.

유형	좌표	gesture 컬럼	전처리 후
손 미검출 (NaN)	빈 값	0	행 제거 (`drop_missing=True`)
Hard negative (손 있음, 제스처 아님)	값 있음	0	피처 계산 후 학습에 포함

hard negative 샘플은 "relaxed hand vs 실제 제스처"를 구분하는 핵심 데이터이므로 제거하면 안 된다.

2. 먼저 잡아야 하는 전제

현재 모델 비교 전에 라벨 정의를 먼저 정리하는 것이 맞다.

- `data/labeling_tool/config/label_config.yaml`은 `0~6`의 gesture class를 정의한다.
- 그런데 `data/landmark_extractor/README.md`의 파일명 예시는 `1_slow_right_woman1.mp4` 형식이다.
- 현재 `src/dataset/gesture_dataset.py`는 파일명의 두 번째 토큰(`split('_')[1]`)을 라벨로 읽는다.
- 그래서 지금 상태로는 `Fist/Open Palm/V/...`를 학습하는 대신 `slow/fast/neutral/hardneg`를 학습할 가능성이 높다.

즉, 모델 비교 실험 전에는 아래 둘 중 하나로 맞추는 것을 권장한다.

1. 파일명 첫 토큰(`0~6`)을 gesture id로 사용
2. `labeled_data` 또는 `total_data`의 `gesture` 컬럼을 정식 라벨로 사용

전처리 파이프라인은 CSV의 `gesture` 컬럼을 직접 사용하므로, 옵션 2를 택하면 라벨 이슈가 자동 해결된다.

3. 추천 모델군 한눈에 보기

모델	입력	Pretrained	장점	약점	이 프로젝트 추천도
MLP baseline	`63` 또는 `156`	아니오	가장 빠르고 구현이 단순함	시간축 정보가 거의 없음	높음
MLP + embedding	`156 -> 128 -> cls`	아니오	작은 비용으로 표현력 상승	여전히 시계열 표현은 약함	매우 높음
Two-Stream MLP	`63 | 93` (joint + bone+angle)	아니오	엣지 정보를 별도 공간에서 학습 후 결합, 작은 데이터에서도 안정적	단일 MLP 대비 파라미터 약간 증가	**매우 높음**

모델	입력	Pretrained	장점	약점	이 프로젝트 추천도
Transformer + embedding	16 x 156	아니오	프레임 간 관계를 직접 학습	데이터가 작으면 과적합 쉬움	높음
1D CNN / TCN	16 x 156	아니오	로컬 temporal pattern에 강하고 Transformer보다 가벼움	장기 의존성은 제한적	매우 높음
MediaPipe Hand Landmarker	RGB -> 21개 손 랜드마크	예	이미 프로젝트에 맞고 tracking까지 포함	gesture classifier 자체는 아님	필수 프론트엔드
MobileNetV3-Small	손 crop 이미지	예	작은 크기 대비 균형이 좋음	이미지 데이터셋을 따로 구성해야 함	조건부 높음
ShuffleNetV2 x0.5	손 crop 이미지	예	매우 가볍고 빠름	정확도 여유가 적음	조건부 중간
EfficientNet-B0	손 crop 이미지	예	작은 축에서는 정확도 우선 카드	MobileNet보다 무거움	조건부 높음

핵심은 다음이다.

- 랜드마크 기반을 유지한다면 Two-Stream MLP, MLP + embedding, 1D CNN/TCN, Transformer가 주력이다.
- **Two-Stream 구조는 bone/angle 피처를 별도 스트림으로 학습시켜 joint 정보와 결합하므로, 단순 concat보다 안정적인 개선을 기대할 수 있다.**
- 이미지 기반으로 갈 때만 MobileNetV3-Small, ShuffleNetV2, EfficientNet-B0가 의미가 있다.
- MediaPipe Hand Landmarker는 분류기 대체재가 아니라, 현재 파이프라인의 가장 적합한 손 검출/추적 프론트엔드다.

3.1 피처 조합 × 모델 실험 매트릭스 (신규)

"bone/angle 피처가 실제로 얼마나 도움이 되는가"를 아키텍처별로 분리해서 보기 위한 매트릭스다.

	joint (63d)	joint+bone (147d)	joint+bone+angle (156d)	two-stream (63 | 93)
MLP baseline	✅ 필수 기준선	✅	✅	—
MLP + embedding	✅	✅	✅	—
Two-Stream MLP	—	—	—	✅ 핵심 실험
1D CNN / TCN	✅	✅	✅	—
Transformer	✅	✅	✅	—

실험 시 유의사항:

- **joint (63d) 열**: 기존 문서의 모든 모델이 이 조건이다. 반드시 먼저 돌려서 기준선을 잡는다.
- **joint+bone+angle (156d) 열**: 단순 concat으로 피처를 넣는 방식. 가장 구현이 간단하다.
- **two-stream 열**: joint와 bone+angle을 각각 별도 MLP로 인코딩한 뒤 late fusion하는 방식. 스켈레톤 행동인식 분야(2s-AGCN 등)에서 반복적으로 효과가 검증된 구조다.
- concat(156d) vs two-stream(63|93) 비교가 "별도 스트림의 가치"를 측정하는 핵심 실험이다.

4. 공통 평가 체계 (신규)

모든 모델 실험에 공통 적용한다. accuracy만으로는 위험하다.

class 0(none/hardneg)이 전체 데이터의 상당 부분을 차지하므로, accuracy는 "전부 0으로 예측"만 해도 쉽게 높아진다. 따라서 아래 지표를 모두 기록해야 공정한 비교가 가능하다.

4.1 필수 지표

지표	의미	비고
Macro F1	클래스별 F1의 단순 평균	**주 비교 지표**로 사용
클래스별 Precision / Recall / F1	어떤 제스처가 잘 안 잡히는지 진단	특히 소수 클래스 recall 주시
Confusion Matrix	어떤 제스처 쌍이 혼동되는지	개선 방향 설정에 필수
Class 0 False Positive Rate	실제 제스처를 none으로 놓치는 비율	실서비스 오작동 지표
Class 0 False Negative Rate	none인데 제스처로 오판하는 비율	실서비스 오작동 지표
추론 시간 (ms/frame)	온디바이스 배포 판단 근거	CPU 기준으로 측정

4.2 불균형 대응 기본 설정

모든 학습 실험에 아래를 기본 적용한다.

- **Loss**: Focal Loss (`gamma=2.0`, class-weight alpha 적용)
- **Sampling**: WeightedRandomSampler (inverse frequency 기반)
- **Split**: `source_file` 기준 GroupShuffleSplit (80/10/10) — 같은 영상의 프레임이 train/val/test에 섞이지 않도록

Focal Loss의 gamma는 하이퍼파라미터이므로, 필요하면 `0, 1, 2, 3`을 sweep할 수 있다. MediaPipe Model Maker도 focal loss gamma를 튜닝 노브로 노출하고 있어 실무적으로 검증된 전략이다.

4.3 결과 기록 템플릿

각 실험의 결과를 아래 형태로 통일한다.

```
실험명: MLP_baseline_joint63
피처: joint (63d)
모델: MLP 63→128→64→7
Loss: FocalLoss(gamma=2.0)
Epochs: 50 (early stop at 32)

| Class     | Precision | Recall | F1    | Support |
|-----------|-----------|--------|-------|---------|
| none      | 0.xxx     | 0.xxx  | 0.xxx | xxxx    |
| Fist      | 0.xxx     | 0.xxx  | 0.xxx | xxx     |
| ...       |           |        |       |         |
| macro avg | 0.xxx     | 0.xxx  | 0.xxx |         |

Class 0 FP Rate: x.x%
Class 0 FN Rate: x.x%
Inference: x.xx ms/frame (CPU)
```

5. 모델별 상세 추천

5.1 MLP baseline

가장 먼저 돌려야 하는 기준선이다.

- 입력: 단일 frame의 `63-dim` (joint only) 또는 `156-dim` (전처리 전체)
- 장점: 학습/추론이 가장 빠르고, 현재 `src/models/baseline/mlp_classifier.py`와도 바로 연결 가능
- 단점: 손 모양은 잡아도 motion pattern은 거의 못 잡음
- 추천 상황: 라벨 정합성 확인, feature 정규화 검증, 작은 데이터셋 sanity check

권장 형태:

- `D -> 128 -> 64 -> 7` (D = 63 또는 156)
- `dropout 0.1~0.3`
- **전처리 파이프라인의 정규화를 반드시 적용** (raw 좌표 직접 투입 금지)

실험 포인트: joint(63) vs joint+bone+angle(156) 비교로 "bone/angle 피처의 가치"를 빠르게 확인할 수 있다.

5.2 MLP + embedding

현재 저장소에서 가장 먼저 성능 상승을 기대할 수 있는 형태다.

- embedding은 여기서 `nn.Embedding` 같은 discrete token embedding이 아니라, `D차원 연속 landmark를 잠재공간으로 투영하는 learnable projection`을 뜻한다.
- 작은 데이터에서는 Transformer보다 안정적으로 잘 붙는 경우가 많다.

예시:

```python
import torch.nn as nn


class MLPWithLandmarkEmbedding(nn.Module):
    def __init__(self, input_dim=156, embed_dim=128, num_classes=7):
        super().__init__()
        self.embedding = nn.Sequential(
            nn.Linear(input_dim, embed_dim),
            nn.LayerNorm(embed_dim),
            nn.GELU(),
            nn.Dropout(0.1),
        )
        self.classifier = nn.Sequential(
            nn.Linear(embed_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, num_classes),
        )

    def forward(self, x):
        x = self.embedding(x)
        return self.classifier(x)
```

추천 이유:

- 현재 데이터 크기에서 가장 리스크가 낮다.
- 전처리 파이프라인의 정규화가 들어가면 landmark quality 편차가 이미 완화되어 있다.
- front-end가 JS이든 Python이든 배포가 편하다.

5.3 Two-Stream MLP (신규)

스켈레톤 행동인식 분야(2s-AGCN)에서 검증된 "joint stream + bone stream → late fusion" 구조를 손 제스처에 맞춘 것이다.

- Stream-J: 정규화 좌표 `63d` → MLP → `embedding_j`
- Stream-B: bone vector+length+angle `93d` (84+9) → MLP → `embedding_b`
- Fusion: concat → MLP head → 7 classes

```python
```

```python
import torch
import torch.nn as nn


class StreamMLP(nn.Module):
    def __init__(self, input_dim, hidden_dims=(128, 128), dropout=0.3):
        super().__init__()
        layers = []
        prev = input_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        self.net = nn.Sequential(*layers)
        self.out_dim = hidden_dims[-1]

    def forward(self, x):
        return self.net(x)


class TwoStreamMLP(nn.Module):
    def __init__(self, joint_dim=63, bone_dim=93, num_classes=7,
                 hidden=(128, 128), fusion=(128, 64), dropout=0.3):
        super().__init__()
        self.joint_stream = StreamMLP(joint_dim, hidden, dropout)
        self.bone_stream = StreamMLP(bone_dim, hidden, dropout)

        fusion_in = self.joint_stream.out_dim + self.bone_stream.out_dim
        layers = []
        prev = fusion_in
        for f in fusion:
            layers += [nn.Linear(prev, f), nn.BatchNorm1d(f), nn.ReLU(), nn.Dropout(dropout)]
            prev = f
        layers.append(nn.Linear(prev, num_classes))
        self.head = nn.Sequential(*layers)

    def forward(self, joint_input, bone_input):
        j = self.joint_stream(joint_input)
        b = self.bone_stream(bone_input)
        return self.head(torch.cat([j, b], dim=-1))
```

핵심 차이:

- 단순 concat(156d → single MLP)은 bone 피처가 joint 피처에 묻힐 수 있다.
- Two-Stream은 각 스트림이 **별도 표현 공간에서 학습된 뒤 결합**되므로, 데이터가 작아도 안정적으로 개선되는 경향이 있다.

- `concat(156d) MLP` vs `Two-Stream(63|93) MLP` 비교가 "별도 스트림의 가치"를 측정하는 핵심 실험이다.

5.4 Transformer + embedding

손 모양 자체보다 "동작의 시간 흐름"이 중요해지면 이쪽이 맞다.

- 입력: `B x T x D`, 예를 들어 `B x 16 x 156`
- embedding은 각 frame을 `D -> d_model`로 투영하는 층이다.
- 그 뒤 positional encoding 또는 learnable positional embedding을 더해 transformer encoder에 넣는다.

예시:

```python
import torch
import torch.nn as nn


class LandmarkTransformer(nn.Module):
    def __init__(self, seq_len=16, input_dim=156, d_model=128,
                 num_heads=4, num_layers=2, num_classes=7):
        super().__init__()
        self.frame_embedding = nn.Linear(input_dim, d_model)
        self.pos_embedding = nn.Parameter(torch.zeros(1, seq_len, d_model))

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=num_heads,
            dim_feedforward=256,
            dropout=0.1,
            batch_first=True,
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)
        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, num_classes),
        )

    def forward(self, x):
        h = self.frame_embedding(x) + self.pos_embedding[:, : x.size(1)]
        h = self.encoder(h)
        pooled = h.mean(dim=1)
        return self.head(pooled)
```

추천 상황:

- `slow`, `fast`처럼 속도 차이까지 gesture identity에 영향을 주는 경우
- frame 단위보다 window 단위 예측이 필요한 경우
- 향후 양손 또는 longer context를 붙일 계획이 있는 경우

주의:

- 현재 데이터 규모에서는 `d_model=128`, `num_layers=2`, `num_heads=4` 정도로 작게 시작하는 편이 맞다.
- 라벨이 frame 단위로 noisy하면 Transformer가 오히려 더 흔들릴 수 있다.

5.5 1D CNN / TCN

실무적으로는 이 프로젝트에 상당히 잘 맞을 가능성이 높다.

- 입력: `16 x D` landmark sequence (D = 63, 147, 또는 156)
- `Conv1d`로 시간축을 따라 지역 패턴을 잡는다.
- Transformer보다 훨씬 가볍고, MLP보다 temporal bias가 강하다.

추천 이유:

- 데이터가 아주 크지 않아도 잘 붙는다.
- 짧은 제스처 구간을 잡는 데 유리하다.
- 모바일/웹 추론으로 옮기기 쉬운 편이다.

한 줄 추천:

- landmark-only 실험에서는 `MLP + embedding` 다음 카드로 `1D CNN/TCN`을 가장 추천한다.

6. 작은 pretrained hand tracking 모델 추천

6.1 1순위: MediaPipe Hand Landmarker

현재 프로젝트와 가장 잘 맞는 선택이다.

- 공식 문서 기준 Hand Landmarker는 `palm detection model + hand landmark detection model`이 묶인 bundle이다.
- video/live-stream 모드에서는 palm detection을 매 frame 돌리지 않고, landmark 기반 tracking이 유지되는 동안 detection을 건너뛰어 비용을 줄인다.
- 공식 overview 기준 `HandLandmarker (full)`의 전체 파이프라인 평균 latency는 `Pixel 6 CPU 17.12ms / GPU 12.27ms`다.
- 공식 모델 정보 기준 입력 shape는 `192 x 192, 224 x 224`, quantization type은 `float16`이다.

이 프로젝트에서의 해석:

- 이미 `hand_landmarker.task`를 보유 중이므로 가장 즉시성 있는 pretrained choice다.
- 분류 모델을 바꾸더라도 hand tracking front-end는 그대로 유지하는 편이 합리적이다.

6.2 보조 옵션: MediaPipe Gesture Recognizer

완전한 end-to-end 시작점을 빨리 보고 싶으면 검토할 가치가 있다.

- Gesture Recognizer는 hand landmarks와 gesture category를 함께 반환한다.
- 공식 문서상 custom model 사용도 지원한다.
- 다만 현재 프로젝트의 label set(`Neutral`, `Fist`, `Open Palm`, `V`, `Pinky`, `Animal`, `K-Heart`)과 기본 canned gesture set이 정확히 일치하지 않을 수 있다.

결론:

- tracking/front-end용 pretrained 모델은 `Hand Landmarker`를 유지
- 빠른 데모용 end-to-end baseline은 `Gesture Recognizer`를 별도 POC로만 비교

7. 이미지 기반 CNN 후보 비교

주의: 아래 모델들은 `현재 CSV landmark 파이프라인의 drop-in replacement`가 아니다.

- 손 crop 이미지 데이터셋을 별도로 만들 수 있을 때 추천한다.
- 즉, `raw_data`에서 손 bbox를 잘라 이미지 분류로 가거나,
- `landmark + image`의 2-branch 모델로 확장할 때 의미가 있다.

7.1 MobileNetV3-Small

TorchVision 공식 문서 기준:

- `acc@1`: `67.668`
- `num_params`: `2,542,856`
- `GFLOPS`: `0.06`
- `file size`: `9.8 MB`

추천 이유:

- 작은 크기 대비 균형이 가장 좋다.
- 손 crop 기반 gesture image classification을 처음 붙일 때 1순위다.

7.2 ShuffleNetV2 x0.5

TorchVision 공식 문서 기준:

- acc@1: 60.552
- num_params: 1,366,792
- GFLOPS: 0.04
- file size: 5.3 MB

추천 이유:

- 극단적으로 가볍게 가야 할 때 좋다.
- landmark 추출 없이 매우 작은 image classifier를 얹고 싶은 경우 후보가 된다.

주의:

- 정확도 여유가 적어서 custom gesture 7종 분류에서 class boundary가 애매하면 먼저 무너질 가능성이 있다.

7.3 EfficientNet-B0

TorchVision 공식 문서 기준:

- acc@1: 77.692
- num_params: 5,288,548
- GFLOPS: 0.39
- file size: 20.5 MB

추천 이유:

- 위 세 개 중 accuracy 쪽으로 가장 무게를 둔 선택이다.
- landmark 품질이 흔들리고 image texture가 중요할 때 더 유리할 수 있다.

주의:

- MobileNetV3-Small보다 무겁다.
- 이 프로젝트의 현재 landmark-first 구조에는 과한 카드일 수 있다.

8. 내가 권장하는 실험 순서

A. 현재 파이프라인 유지 (업데이트)

```
Phase 0 — 전제 조건
  ├── 라벨 추출 방식 수정 (gesture 컬럼 사용)
  └── hand_gesture_preprocess.py 실행 → 전처리 CSV 생성

Phase 1 — 기준선 (정적/단일 프레임)
  ├── ① MLP baseline + joint(63d)        ← 최소 기준선
```

```
  ├── ② MLP baseline + joint+bone+angle(156d) ← bone/angle 가치 확인
  ├── ③ MLP + embedding + 156d          ← embedding 효과 확인
  └── ④ Two-Stream MLP (63|93)          ← 별도 스트림 가치 확인

Phase 2 — 시계열 확장
  ├── ⑤ 1D CNN/TCN + 156d (16 frames)
  └── ⑥ Transformer + embedding + 156d (16 frames)

Phase 3 — 비교 분석
  └── 섹션 4 기준 통일 지표로 전 모델 비교표 작성
```

핵심 비교 포인트:

- **② vs ①**: bone/angle 피처를 넣으면 얼마나 좋아지나?
- **④ vs ②**: 같은 피처를 concat vs two-stream으로 넣으면 차이가 있나?
- **⑤⑥ vs ④**: 시계열을 보면 정적 모델 대비 얼마나 좋아지나?

B. landmark 품질이 병목일 때

1. `MediaPipe Hand Landmarker` 유지
2. `raw_data`에서 손 crop 이미지셋 생성
3. `MobileNetV3-Small` 실험
4. 필요 시 `ShuffleNetV2 x0.5` 또는 `EfficientNet-B0` 비교

9. 최종 추천 요약

가장 현실적인 조합은 아래다.

추천 1 (최우선)

- `Hand Landmarker + 전처리 정규화 + Two-Stream MLP`
- 이유: bone/angle 피처를 가장 효과적으로 활용하면서도 작은 데이터에서 안정적

추천 2

- `Hand Landmarker + 전처리 정규화 + MLP + embedding`
- 이유: Two-Stream 대비 구현이 더 단순하고, 단일 입력 파이프라인으로 배포가 간편

추천 3

- `Hand Landmarker + 1D CNN/TCN`
- 이유: temporal 정보가 필요한데 Transformer까지는 과할 수 있는 경우

추천 4

- `Hand Landmarker + Transformer + embedding`
- 이유: 동작 길이, 속도 차이, 양손 확장까지 고려한 중장기 방향

추천 5

- `Hand Landmarker + hand crop image + MobileNetV3-Small`
- 이유: landmark만으로 모양 구분이 부족할 때 가장 실용적인 image backbone

10. 소스

프로젝트 내부 참고:

- `src/models/baseline/mlp_classifier.py`
- `src/dataset/gesture_dataset.py`
- `src/dataset/sliding_window.py`
- `data/labeling_tool/config/label_config.yaml`
- `data/landmark_extractor/README.md`
- `hand_landmarker.task`
- `hand_gesture_preprocess.py` (전처리 파이프라인, 신규)
- `hand_gesture_pipeline.py` (전체 학습 파이프라인, 신규)

공식 문서:

- MediaPipe Hand Landmarker Python guide: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python
- MediaPipe Hand Landmarker overview: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- MediaPipe Gesture Recognizer: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
- TorchVision MobileNetV3-Small: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.mobilenet_v3_small.html
- TorchVision ShuffleNetV2 x0.5: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.shufflenet_v2_x0_5.html
- TorchVision EfficientNet-B0: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.efficientnet_b0.html

참고 논문:

- 2s-AGCN (Two-Stream Adaptive Graph Convolutional Networks): joint + bone two-stream 융합의 근거
- ST-GCN (Spatial Temporal Graph Convolutional Networks): 스켈레톤 시공간 그래프 학습의 기초
- MediaPipe 온디바이스 HGR 논문: 정규화, focal loss, 2단계 파이프라인 설계의 근거

참고:

- image backbone의 `acc@1` 수치는 ImageNet-1K 기준이므로, 현재 gesture 데이터셋 성능을 직접 의미하지는 않는다.
- 실험 우선순위와 최종 추천은 위 소스와 현재 저장소 구조를 바탕으로 한 프로젝트 맞춤 추론이다.